

ComponentGraph vs ComputationGraph

Updated 22 September 2026

Comparison of the current agentofreality-parallel-computation-graph branch, not a published release.

Bottom line

ComputationGraph is a working alternative with better control over components and more flexible processing arrangements. It is not yet a faster replacement.

Independent queries now run in separate, owned tasks. The last measured ComputationGraph run took **23.1% longer than ComponentGraph** on that workload. Those timings predate the recovery changes below and have not been rerun for them.

Continue developing it, but keep ComponentGraph as the default for now.

What both engines already support

Both use the same underlying query-evaluation code. Both support the existing source, query and reaction interfaces, Cypher/GQL, joins, middleware, initial data loading, scheduled work and recovery where the source and storage support it. Both engines use the same plugin interface. The current SDK contract is 0.15.0; locally built hosts and plugins must use matching SDK versions.

These improvements apply to **both engines**, not just ComputationGraph:

- Queued query inputs are ordered by source-reported event time, then source order in the query definition, then source-maintained sequence number. This does not guarantee an order over events that have not arrived.
- Source events require a sequence number. Concurrent application, HTTP and gRPC ingestion preserves saved-event order through delivery. Source bootstrappers stream records rather than collecting a complete snapshot before sending it.
- Server reports a loaded plugin's actual package version, not its configuration-format version.

The meaningful differences

Area	ComponentGraph	ComputationGraph
Who manages execution	The graph records components and status; source, query and reaction managers run them.	The graph owns components and connections and decides when to create, start, stop or replace them.
Processing arrangements	Built around source-to-query-to-reaction pipelines, with middleware for preprocessing.	Also supports custom transformers, sinks and services, including paths with no query.
Stateful middleware recovery	Middleware inside a persistent query uses that query's saved element state.	Standalone middleware can atomically save its own element state, input position and pending output, with durable connections.
Connection checks	Checks follow the established component interfaces and dependencies.	Components declare named inputs/outputs and required formats and delivery behaviour.
Adding a component	Addition can wait for initialization; a failed source initialization removes its entry.	An accepted node appears first. Later initialization/startup failures stay visible on it.
Readiness	Existing status and start methods.	Separate handles can wait for creation or actual readiness; accepting an addition is not readiness.

Area	ComponentGraph	ComputationGraph
Changing a running system	Component-specific update and removal APIs.	Can preview changes and reconnect affected components. Old handles cannot control replacement instances.
Inspection	Established component status, relationships and events.	Also shows nested query graphs, known resources, plugin references and recent graph changes.
Readiness / cleanup	Managed through component managers and plugins.	Separate readiness messages and explicit cleanup ownership.

The standard DrasiLib and Server APIs can select either engine. A DrasiLib instance can also host additional computation graphs alongside its existing ComponentGraph pipeline. Neither arrangement automatically migrates saved state.

Last measured performance

The latest controlled comparison used **100,000 inputs and two Building Comfort queries**: room results and floor aggregation. Each engine had one warm-up and five measured runs, with alternating engine order and the same release binary.

Engine	Median elapsed time	Range across measured runs
ComponentGraph	3.804 seconds	3.781-3.877 seconds
ComputationGraph	4.681 seconds	4.629-4.719 seconds

The scheduling fix reduced ComputationGraph's median elapsed time from **7.152 to 4.681 seconds**, a **34.5% reduction**. Queries previously shared one execution task; their separate tasks now allow independent queries to use different CPU workers.

Per-change timing and CPU samples show that converting source events and query results between formats remains a substantial cost. Query timings now include commit. Timing intervals can include waiting and are not pure CPU measurements.

Every run matched 99,981 room results and 49,860 floor results, including ordered result hashes. The comparison checks result values and their order; changing timestamps and delivery bookkeeping are excluded.

The measurement window starts at the first observed result and ends when both full result streams finish. It excludes startup/shutdown but includes generation, queues, result collection and hashing. Profiling-file output was disabled during timing runs.

This was an in-memory, in-process workload, without Server, network transports or dynamically loaded plugins. It is evidence for this workload, not proof of equal behaviour or performance in every deployment.

Recovery and operational behaviour

The recovery audit identified real differences, not just missing tests. The following changes address them:

- A persistent query with no saved position for one source requests replay from the beginning, without clearing other sources' committed state. That zero position is preserved through dynamic plugin calls.
- Reaction AutoSkipGap applies to changed/recreated queries as well as missing retained results. Strict still refuses unsafe continuation; AutoReset uses a supported replacement snapshot.
- Custom replay helpers check producer identity and missing live sequences. Matching query names or sequence numbers alone cannot authorize reuse of a checkpoint from another instance.

- Durable standalone middleware restores Unwind's previous elements and resends unconfirmed saved output without running the middleware twice. Every outgoing branch must accept durably before confirmation; full retention cannot silently discard pending output. Persistent queries reject known volatile middleware.
- Reaction snapshot streams avoid converting the whole result set to JSON. Encoded rows are still validated in a constant-memory scan before rows are converted on demand.

Query state, source checkpoints, output sequence and retained results can commit together with a supporting storage provider. Reactions save completed handling, not just queue acceptance. A crash after an external action but before its checkpoint can repeat that action; neither engine promises exactly-once effects. Corrupt saved output follows the configured recovery policy rather than failing with an unclassified decoding error. Output-store read outages do not authorize clearing state.

Dynamically loaded sources can replay when their plugin supports it. Sources without replay or a retained event log cannot resend lost data. Persistent query indexes alone do not make query output durable.

Conformance scenarios select the execution engine explicitly, including child processes. Coverage includes abrupt exits around reaction checkpoints and middleware delivery, partial fanout, saved state reopening and configured recovery policies. It is evidence for those paths, not proof about every third-party plugin.

Main remaining work

- **Reduce the remaining performance gap** and measure more queries, multiple sources, persistent storage and real plugins.
- **Create source/reaction plugins from configuration inside the standard add APIs.** They currently receive preconstructed objects, so constructor failures before the call cannot appear as failed graph nodes.
- **Record all provider relationships and creation details.** Graph export does not yet capture everything needed to recreate every plugin, storage provider and connection to a supplied object.
- **Move more deployment/cloning work into reusable library APIs** and expose the richer graph controls through Server.
- **Decide whether all-or-nothing batch deployment is needed.** Additions deliberately remain visible, including failures. Saving one query's storage updates together does not make a whole deployment all-or-nothing.

Versions and supporting material

Recovery checkpoints are core 15eaaa04 (sources), 4110b3ee (consumers) and 31f9b9be (middleware). This revision adds explicit crash conformance and corrects saved-output corruption handling. Server remains at 58669661 and test-infra at d3fbbbb4. Migration between engines is outside this recovery work. The performance figures are from the earlier core commit 6a6301bc, not these recovery revisions. Benchmark evidence is retained in this workspace session's `parallel-performance-summary.json`, `ranked-baseline/` and `parallel-query-benchmark/` records.

For details, use the [design overview](#), [usage guide](#) and [configuration reference](#).